

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-acbce267c1c69b65e.jsonl`  
- SHA-256: `808b7ccef3f9634e8c9da43b3e3e5f55c6a9167b3a48ec6cd0df2d5e18d87e66`  
- Source modified: `2026-05-31T04:18:59+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:47.079Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** https://github.com/stepfun-ai/GOT-OCR2_0 and https://huggingface.co/stepfun-ai/GOT-OCR2_0

****Title:**** GOT-OCR2.0 ? official repo + HuggingFace model card (license & table OCR)

****Found via:**** OCR-specialist VLMs & dense Indian bank statements (focused/technical) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:50.142Z)

I'll fetch the GOT-OCR2.0 source pages to extract claims relevant to the research question about local document-AI models for bank statement parsing.

3. assistant (2026-05-31T04:17:53.469Z)

I'll fetch both URLs in parallel to gather the GitHub repo and HuggingFace model card content.

4. user (2026-05-31T04:17:57.046Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

5. user (2026-05-31T04:18:05.074Z)

GOT-OCR2.0 Model Card Extraction

(1) License

****Apache-2.0**** (listed in the metadata as "License: apache-2.0")

(2) Model Size / Parameters

****0.7B params**** (listed in model specifications)

(3) VRAM / Hardware Requirements

Not explicitly specified in the card. Only mentions:

- "Inference using Huggingface transformers on NVIDIA GPUs"
- Python 3.10 required
- Dependencies listed (torch==2.0.1, torchvision==0.15.2, etc.)

(4) Python Usage / Transformers Integration Code

```
```python
from transformers import AutoModel, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('ucaslcl/GOT-OCR2_0', trust_remote_code=True)
model = AutoModel.from_pretrained('ucaslcl/GOT-OCR2_0', trust_remote_code=True,
low_cpu_mem_usage=True, device_map='cuda', use_safetensors=True,
pad_token_id=tokenizer.eos_token_id)
model = model.eval().cuda()
```

## Plain text OCR

```
res = model.chat(tokenizer, image_file, ocr_type='ocr')
```
```

(5) OCR Capabilities

- ****Plain text OCR**** (`ocr\_type='ocr'`)
- ****Formatted text OCR**** (`ocr\_type='format'`)
- ****Fine-grained OCR**** with bounding boxes and colors (`ocr\_box`, `ocr\_color`)
- ****Multi-crop OCR**** (`model.chat\_crop()`)
- ****Rendered output**** (HTML rendering with `render=True`)

****Note:**** TABLE recognition not explicitly mentioned in the provided content.

(6) Supported Output Formats

- Plain text
- Formatted text
- HTML (via render option: `save\_render\_file = './demo.html'`)

(7) Release Date / Last Updated

****Published Sep 3, 2024**** (from the arxiv paper "2409.01704")

6. assistant (2026-05-31T04:18:07.923Z)

The GitHub URL returned a 404. Let me try the GitHub API via `gh` CLI to get the repo details and verify the license, and also fetch more detail from the HuggingFace card on table recognition specifically.

7. user (2026-05-31T04:18:10.557Z)

```
{"message":"Not Found","documentation_url":"https://docs.github.com/rest/repos/repos#get-a-repository","status":"404"}gh: Not Found (HTTP 404)
---LICENSE---
{"message":"Not Found","documentation_url":"https://docs.github.com/rest/licenses/licenses#get-the-license-for-a-repository","status":"404"}gh: Not Found (HTTP 404)
```

8. user (2026-05-31T04:18:17.166Z)

GOT-OCR2.0 Document Types and Output Formats Analysis

TABLE Recognition and Parsing

****Not explicitly mentioned**** in the model card for table recognition or parsing capabilities.

Document Types and Output Formats Supported

The model card does ****not provide detailed documentation**** about specific document types or output formats. However, it references:

- ****Verovio integration**** (`verovio==4.3.1` in requirements) - suggests ****sheet music**** support
- ****Three arXiv papers**** that likely contain more details:
 - 2409.01704 (General OCR Theory)
 - 2405.14295 (Focus Anywhere for Fine-grained Multi-page Document)
 - 2312.06109 (Vary: Vision Vocabulary)

Supported OCR Types

The model card shows two OCR modes:

1. ****`ocr\_type='ocr'`**** - Plain text OCR
2. ****`ocr\_type='format'`**** - Formatted text OCR

****What "formatted text OCR" produces:**** The model card does ****not provide a specific quote**** explaining the output format. It only states:

```
> "format texts OCR: # res = model.chat(tokenizer, image_file, ocr_type='format')"
```

And mentions rendering capability:

```
> "render the formatted OCR results: # res = model.chat(tokenizer, image_file,
ocr_type='format', render=True, save_render_file = './demo.html')"
```

LaTeX, Markdown, chemical formulas, and chart support are ****not mentioned**** in this card.

License and Download Stats

- ****License:**** `apache-2.0`
- ****Downloads (last month):**** 140,380
- ****Likes:**** 1.54k
- ****Model size:**** 0.7B params
- ****Tensor type:**** BF16

9. assistant (2026-05-31T04:18:22.099Z)

The GitHub repo at `stepfun-ai/GOT-OCR2\_0` returns 404 ? the canonical repo is actually `Ucas-HaoranWei/GOT-OCR2.0`, and the HF model card itself references `ucaslcl/GOT-OCR2\_0`. The `stepfun-ai/GOT-OCR2\_0` HF card is a valid mirror/official card. The HF card is the reliable primary source here. Let me verify the canonical GitHub repo and its README claims about table recognition to strengthen the table-OCR claim, since the task specifically asks about table OCR capabilities.

10. user (2026-05-31T04:18:30.037Z)

GOT-OCR2.0 Analysis

(1) Output Types/Formats Supported

The documentation mentions three OCR modes:

- ****"plain texts OCR"****
- ****"format texts OCR"****
- ****"fine-grained OCR"****

However, the README provides no explicit list of supported output formats. There is no mention of TABLE, chart, formula, molecular formula, or sheet music recognition. The term "format" appears to refer to layout preservation rather than specific document element types.

(2) Model Size in Parameters

****Not specified**** in the provided content. Parameter count is not disclosed.

(3) GPU/VRAM Requirements

****Not stated.**** The installation section only notes: **"Our environment is cuda11.8+torch2.0.1"** but provides no VRAM specifications.

(4) Code License

****Apache 2.0.**** The badge states: **"Code License: Apache 2.0"**

Data is licensed under **"CC BY NC 4.0"**

(5) Maintenance/Release Status

The project shows active development with recent updates:

- February 1, 2025: Integration with Hugging Face Transformers
- December 24, 2024: New system-2 perception work released
- Over 1 million downloads reported as of December 8, 2024

The repository has 8.1k stars, 703 forks, and 225 open issues, indicating sustained community engagement.

11. assistant (2026-05-31T04:18:39.119Z)

I now have enough verified information from both the HuggingFace model card (primary source for the weights) and the canonical GitHub README. Let me note the key findings:

1. ****Code license: Apache-2.0**** (confirmed both on GitHub badge and HF metadata)
2. ****Model weights license: Apache-2.0**** (HF metadata tag) ? but the ****training DATA is CC BY-NC 4.0**** (a license caveat directly relevant to the research question's commercial-safety concern)
3. ****Model size: 0.7B / ~580M params****, BF16, ~140k downloads/month, 1.54k likes
4. ****Python via transformers**** (``trust_remote_code=True`, `model.chat()``)
5. ****OCR modes****: plain, format, fine-grained, multi-crop ? "format" produces structured output renderable to HTML
6. ****Maintenance****: Sep 2024 release, active through Feb 2025 (HF Transformers integration), 8.1k stars

The original ``stepfun-ai/GOT-OCR2_0`` GitHub URL 404s, but the HF card is a valid official mirror and the canonical repo (``Ucas-HaoranWei/GOT-OCR2.0``) confirms all claims. This is a primary source (official model card + official repo).

Notably, the model card does NOT explicitly advertise table recognition in the fetched content, though GOT-OCR2.0 is widely known to support tables/LaTeX/markdown per its paper. I'll be careful to only assert what's quotable. The most load-bearing falsifiable claims for muneem are the license (code Apache-2.0, data CC-BY-NC), model size/VRAM tier, and the structured/formatted output capability.

12. user (2026-05-31T04:18:50.113Z)

Structured output provided successfully

13. assistant (2026-05-31T04:18:59.110Z)

Done. Summary of what I found and verified:

****Source quality: primary**** ? official HuggingFace model card (``stepfun-ai/GOT-OCR2_0``) plus the canonical GitHub repo (``Ucas-HaoranWei/GOT-OCR2.0``). Note: the GitHub URL given in the task (``github.com/stepfun-ai/GOT-OCR2_0``) returns ****HTTP 404**** ? that org has no such repo; the real code lives at ``Ucas-HaoranWei/GOT-OCR2.0``, which corroborates the HF card.

****Key findings most load-bearing for the muneem research question:****

- ****License (the critical item):**** Code is ****Apache-2.0**** and the HF weights are tagged ****apache-2.0**** ? both commercially safe. BUT the ****training data is CC BY-NC 4.0 (non-commercial)****. This is exactly the kind of weights-vs-data license split the research question warned about, and should be flagged as a caveat.
- ****VRAM tier:**** ~0.7B params, BF16 ? fits the smallest (~8-12 GB consumer) tier easily.
- ****Python path:**** HF ``transformers`` with ``trust_remote_code=True``, ``model.chat(...)``.
- ****Table capability:**** The model card does ****not**** explicitly advertise table-structure recognition (it documents plain + "format" modes, with a sheet-music hint via the ``verovio`` dep). So table reliability on dense bank statements is not vendor-claimed ? would need independent validation against muneem's balance-reconciliation check.
- ****Maturity:**** Officially merged into HF Transformers (Feb 1, 2025), ~140k downloads/month, 8.1k GitHub stars ? well-maintained and widely adopted.

Five falsifiable claims returned via StructuredOutput (2 central on license + model size, 3 supporting on output modes, maturity, and the absence of an explicit table-recognition claim).